

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра САПР

Практическая работа №2
по дисциплине «Объектно-ориентированное программирование»
Тема: «Консольная программа для работы с CSV и XML файлами»

Студент гр. 4354

Кирсова В.В.
Коробцова Д.А.

Преподаватель

Кулагин М.В.

Санкт-Петербург

2025

1. Задание

Необходимо разработать консольное приложение для работы с двумя файлами в формате xml и csv. В результате работы приложения будут выводиться дублирующие записи с количеством повторений, количество 1, 2, 3, 4 и 5 этажных зданий в каждом городе, время обработки файла.

2. Спецификация программы

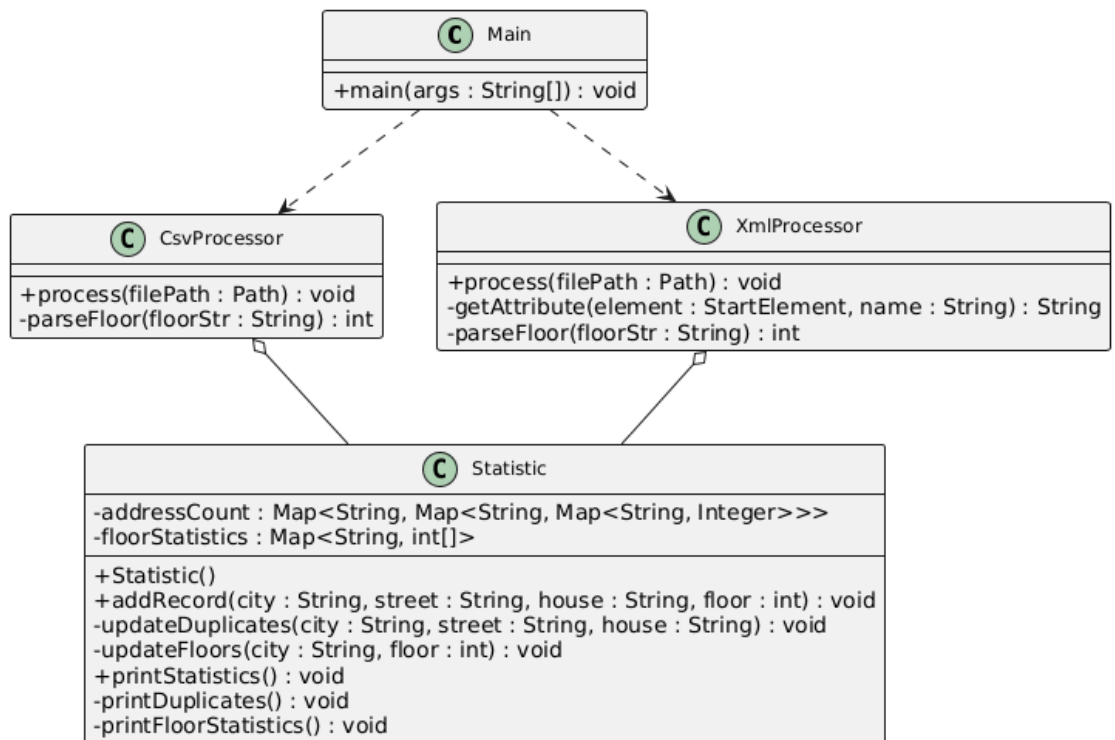


Рис. 1 – Диаграмма классов

3. Описание интерфейса пользователя программы

Outp1 = “Введите путь до файла (csv/xml) или 0 для выхода:”

Inp1 = путь к файлу (или 0). *Пример: "data.csv", "0".*

Outp2 = “Программа завершена.”

Outp3 = “Некорректный путь к файлу.”

Outp3 = “Файл не найден. Попробуйте снова.”

Outp4 = “Неподдерживаемый формат файла.”

Outp5 = “Ошибка чтения файла.”

Outp6 = “Ошибка при разборе XML-файла.”

Outp7 = “Ошибка формата данных: ожидалось число этажа.”

Outp8 = “Время обработки:” <время> “мс\n”.

Пример: Время обработки: 1700 мс.

Outp9 = “Дубликаты”

Outp10 = “Дубликатов не найдено”

Outp11 = “Город:” <название города>, “Улица:” <название улицы>,
“Дом:” <house> - <count> “повторений”.

*Пример: Город: Абакан, Улица: Улица Александровы Пруды, Дом: 1 - 5
повторений*

Outp12 = “Статистика по этажам”

Outp13 = “Данных по этажам не найдено”

Outp14 = “Город” <название города>:

“1-этажных зданий:” <количество_1>

“2-этажных зданий:” <количество_2>

“3-этажных зданий:” <количество_3>

“4-этажных зданий:” <количество_4>

“5-этажных зданий:” <количество_5>

Пример: Город Белорецк:

1-этажных зданий: 9337

2-этажных зданий: 9571

3-этажных зданий: 9289

4-этажных зданий: 9408

5-этажных зданий: 9558

Примеры работы:

```

Введите путь до файла (csv/xml) или 0 для выхода:
/Users/var.kirsova/Downloads/address.xml
Дубликаты

Город: Абакан, Улица: Улица Александровы Пруды, Дом: 1 - 5 повторений

Статистика по этажам
Город Абакан:
  1-этажных зданий: 9355
  2-этажных зданий: 9441
  3-этажных зданий: 9452
  4-этажных зданий: 9484
  5-этажных зданий: 9435
Город Буденновск:
  1-этажных зданий: 9372
  2-этажных зданий: 9671
  3-этажных зданий: 9338
  4-этажных зданий: 9389
  5-этажных зданий: 9393
Город Белорецк:
  1-этажных зданий: 9337
  2-этажных зданий: 9571
  3-этажных зданий: 9289
  4-этажных зданий: 9408
  5-этажных зданий: 9558
Город Арсеньев:
  1-этажных зданий: 9393
  2-этажных зданий: 9430
  3-этажных зданий: 9348
  4-этажных зданий: 9429

```

Рисунок 1

```

  5-этажных зданий: 9428
Город Березники:
  1-этажных зданий: 9330
  2-этажных зданий: 9504
  3-этажных зданий: 9423
  4-этажных зданий: 9468
  5-этажных зданий: 9438
Время обработки: 3385 мс

Введите путь до файла (csv/xml) или 0 для выхода:
/Users/var.kirsova/Downloads/address.csv
Дубликаты

Город: "Абакан", Улица: "Улица Александровы Пруды", Дом: 1 - 5 повторений

Статистика по этажам
Город "Биробиджан":
  1-этажных зданий: 9306
  2-этажных зданий: 9446
  3-этажных зданий: 9484
  4-этажных зданий: 9478
  5-этажных зданий: 9449
Город "Артем":
  1-этажных зданий: 9517
  2-этажных зданий: 9457
  3-этажных зданий: 9235

```

Рисунок 2

4. Текст программы

Main.java

```
import java.util.Scanner;
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import javax.xml.stream.XMLStreamException;

public class Main {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("Введите путь до файла (csv/xml) или 0 для выхода:");
            String input = scanner.nextLine().trim();

            if (input.equals("0")) {
                System.out.println("Программа завершена.");
                break;
            }

            Path filePath;
            try {
                filePath = Path.of(input);
            } catch (java.nio.file.InvalidPathException e) {
                System.out.println("Некорректный путь к файлу.");
                continue;
            }

            if (!Files.exists(filePath)) {
                System.out.println("Файл не найден. Попробуйте снова.");
                continue;
            }
        }
    }
}
```

```

        long startTime = System.currentTimeMillis();

        try {
            if (input.toLowerCase().endsWith(".xml")) {
                XmlProcessor.process(filePath); // передаём Path
            } else if (input.toLowerCase().endsWith(".csv")) {
                CsvProcessor.process(filePath); // передаём Path
            } else {
                System.out.println("Неподдерживаемый формат файла.");
                continue;
            }
        } catch (IOException e) {
            System.out.println("Ошибка чтения файла.");
        } catch (XMLStreamException e) {
            System.out.println("Ошибка при разборе XML-файла.");
        } catch (NumberFormatException e) {
            System.out.println("Ошибка формата данных: ожидалось число этажа.");
        }

        long endTime = System.currentTimeMillis();
        System.out.println("Время обработки: " + (endTime - startTime) + " мс\n");
    }

    scanner.close();
}
}

```

CsvProcessor.java

```

import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class CsvProcessor {

```

```

public static void process(Path filePath) throws IOException {
    Statistic stats = new Statistic();

    try (BufferedReader reader = Files.newBufferedReader(filePath)) {
        String line;
        boolean isFirstLine = true;

        while ((line = reader.readLine()) != null) {
            if (isFirstLine) {
                isFirstLine = false;
                continue;
            }

            String[] parts = line.split(";");
            if (parts.length >= 4) {
                String city = parts[0].trim();
                String street = parts[1].trim();
                String house = parts[2].trim();
                int floor = parseFloor(parts[3].trim());

                stats.addRecord(city, street, house, floor);
            }
        }

        stats.printStatistics();
    }

    private static int parseFloor(String floorStr) {
        try {
            int f = Integer.parseInt(floorStr);
            if (f >= 1 && f <= 5) return f;
        } catch (NumberFormatException ignored) {}
        return 0;
    }
}

```

```
}  
}
```

XmlProcessor.java

```
import java.io.FileInputStream;  
import java.io.IOException;  
import java.nio.file.Path;  
import javax.xml.stream.XMLInputFactory;  
import javax.xml.stream.XMLEventReader;  
import javax.xml.stream.events.XMLEvent;  
import javax.xml.stream.events.StartElement;  
import javax.xml.stream.XMLStreamException;  
  
public class XmlProcessor {  
  
    public static void process(Path filePath) throws IOException, XMLStreamException {  
        Statistic stats = new Statistic();  
  
        XMLInputFactory factory = XMLInputFactory.newInstance();  
  
        try (FileInputStream fis = new FileInputStream(filePath.toFile())) {  
            XMLEventReader reader = factory.createXMLEventReader(fis);  
  
            while (reader.hasNext()) {  
                XMLEvent event = reader.nextEvent();  
  
                if (event.isStartElement()) {  
                    StartElement startElement = event.asStartElement();  
  
                    if ("item".equals(startElement.getName().getLocalPart())) {  
                        String city = getAttribute(startElement, "city");  
                        String street = getAttribute(startElement, "street");  
                        String house = getAttribute(startElement, "house");  
                        int floor = parseFloor(getAttribute(startElement, "floor"));
```



```

        stats.addRecord(city, street, house, floor);
    }
}
}
}

stats.printStatistics();
}

```

```

private static String getAttribute(StartElement element, String name) {
    if (element.getAttributeByName(javax.xml.namespace.QName.valueOf(name)) != null) {
        return
element.getAttributeByName(javax.xml.namespace.QName.valueOf(name)).getValue().trim();
    }
    return "";
}

```

```

private static int parseFloor(String floorStr) {
    try {
        int f = Integer.parseInt(floorStr);
        if (f >= 1 && f <= 5) return f;
    } catch (NumberFormatException ignored) { }
    return 0;
}
}

```

Statistic.java

```
import java.util.*;
```

```

public class Statistic {
    private Map<String, Map<String, Map<String, Integer>>> addressCount;
    private Map<String, int[]> floorStatistics;

```

```

public Statistic() {
    addressCount = new HashMap<>();
    floorStatistics = new HashMap<>();
}

public void addRecord(String city, String street, String house, int floor) {
    updateDuplicates(city, street, house);
    updateFloors(city, floor);
}

private void updateDuplicates(String city, String street, String house) {
    if (city.isEmpty() || street.isEmpty() || house.isEmpty()) {
        return;
    }

    Map<String, Map<String, Integer>> streetsMap = addressCount.get(city);
    if (streetsMap == null) {
        streetsMap = new HashMap<>();
        addressCount.put(city, streetsMap);
    }

    Map<String, Integer> housesMap = streetsMap.get(street);
    if (housesMap == null) {
        housesMap = new HashMap<>();
        streetsMap.put(street, housesMap);
    }

    Integer count = housesMap.get(house);
    if (count == null) {
        housesMap.put(house, 1);
    } else {
        housesMap.put(house, count + 1);
    }
}

```

```
private void updateFloors(String city, int floor) {
    if (city.isEmpty() || floor < 1 || floor > 5) {
        return;
    }
```

```
    int[] floors = floorStatistics.get(city);
    if (floors == null) {
        floors = new int[5];
        floorStatistics.put(city, floors);
    }
```

```
    floors[floor - 1]++;
}
```

```
public void printStatistics() {
    printDuplicates();
    printFloorStatistics();
}
```

```
private void printDuplicates() {
    System.out.println("Дубликаты\n");
    boolean hasDuplicates = false;
```

```
    for (String city : addressCount.keySet()) {
        Map<String, Map<String, Integer>> streets = addressCount.get(city);
```

```
        for (String street : streets.keySet()) {
            Map<String, Integer> houses = streets.get(street);
```

```
            for (String house : houses.keySet()) {
                int count = houses.get(house);
                if (count > 1) {
                    System.out.printf("Город: %s, Улица: %s, Дом: %s - %d повторений\n",
                        city, street, house, count);
```

```

        hasDuplicates = true;
    }
}
}

if (!hasDuplicates) {
    System.out.println("Дубликатов не найдено");
}
}

private void printFloorStatistics() {
    System.out.println("\nСтатистика по этажам");

    if (floorStatistics.isEmpty()) {
        System.out.println("Данных по этажам не найдено");
        return;
    }

    for (String city : floorStatistics.keySet()) {
        int[] floors = floorStatistics.get(city);
        System.out.printf("Город %s:%n", city);

        for (int i = 0; i < floors.length; i++) {
            System.out.printf("  %d-этажных зданий: %d%n", i + 1, floors[i]);
        }
    }
}
}

```

5. Выводы

В результате выполнения практической работы было разработано консольное приложение для обработки данных о зданиях из файлов в форматах CSV и XML. Программа выполняет анализ данных, выводит статистику по дублирующимся записям с количеством повторений, а

также подсчитывает распределение зданий по этажности (от 1 до 5 этажей) для каждого города. Дополнительно фиксируется время обработки каждого файла, что позволяет оценить эффективность работы алгоритмов.

В процессе разработки были освоены ключевые аспекты работы с файлами в Java: обработка CSV-файлов через `BufferedReader` и строковые операции, парсинг XML-документов с использованием StAX-анализатора (`XMLStreamReader`), эффективная организация данных с помощью вложенных коллекций `HashMap` и массивов, а также измерение производительности кода. Проект реализован с применением модульной архитектуры и обработкой исключений. Результат размещён в репозитории на GitHub:

https://github.com/varkirsova/oop_2